

TUGAS PERTEMUAN 11

Nama: Zaky Alnadif

NIM : J0403251126

1. Source code Python

```
8  # Adjacency List
9  graph = {
10     "Bogor": ["Jakarta", "Sukabumi"],
11     "Jakarta": ["Bogor", "Bandung", "Bekasi"],
12     "Bandung": ["Jakarta", "Cirebon"],
13     "Bekasi": ["Jakarta", "Cirebon"],
14     "Cirebon": ["Bandung", "Bekasi", "Sukabumi"],
15     "Sukabumi": ["Bogor", "Cirebon"]
16 }
17
18 # Daftar node
19 nodes = list(graph.keys())
20
21 # Membuat adjacency matrix
22 matrix = []
23
24 for kota1 in nodes:
25     row = []
26
27     for kota2 in nodes:
28         if kota2 in graph[kota1]:
29             row.append(1)
30         else:
31             row.append(0)
32
33     matrix.append(row)
34
35 # Header matrix
36 print("      ", end="")
37 for kota in nodes:
38     print(f"{kota:>10}", end="")
39 print()
40
41 # Isi matrix
42 for i in range(len(nodes)):
43     print(f"{nodes[i]:>10}", end="")
44     for j in range(len(nodes)):
45         print(f"{matrix[i][j]:>10}", end="")
46     print()
47
48 # =====
49 # ANALISIS SINGKAT
50 # =====
51
52 print("\n=== ANALISIS GRAPH ===")
53 print("1. Jenis Graph : Undirected Graph")
54 print("2. Node adalah kota")
55 print("3. Edge adalah jalan penghubung antar kota")
56 print("4. Adjacency List lebih cocok digunakan")
57 print("   karena lebih sederhana dan hemat memori")
58 print("5. Graph termasuk sparse graph")
```

```

35 # =====
36 # OUTPUT PROGRAM
37 # =====
38
39 print("=== ADJACENCY LIST ===")
40 for kota in graph:
41     print(kota, "->", graph[kota])
42
43 print("\n=== NODE (KOTA) ===")
44 for kota in nodes:
45     print("-", kota)
46
47 print("\n=== HUBUNGAN ANTAR KOTA (EDGE) ===")
48 sudah_dicetak = []
49
50 for kota in graph:
51     for tujuan in graph[kota]:
52         # Menghindari edge ganda
53         if (tujuan, kota) not in sudah_dicetak:
54             print(kota, "<->", tujuan)
55             sudah_dicetak.append((kota, tujuan))
56
57 print("\n=== ADJACENCY MATRIX ===")
58
59 # header matrix
60 print(" ", end=" ")
61 for kota in nodes:
62     print(f"({kota}) ", end=" ")
63 print()
64
65 # Isi matrix
66 for i in range(len(nodes)):
67     print(f"({nodes[i]}) ", end=" ")
68     for j in range(len(nodes)):
69         print(f"({nodes[j]}) ", end=" ")
70

```

2. Hasil program

```

PROBLEMS 32 TERMINAL OUTPUT DEBUG CONSOLE PORTS

=== ADJACENCY LIST ===
Bogor -> ['Jakarta', 'Sukabumi']
Jakarta -> ['Bogor', 'Bandung', 'Bekasi']
Bandung -> ['Jakarta', 'Cirebon']
Bekasi -> ['Jakarta', 'Cirebon']
Cirebon -> ['Bandung', 'Bekasi', 'Sukabumi']
Sukabumi -> ['Bogor', 'Cirebon']

=== NODE (KOTA) ===
- Bogor
- Jakarta
- Bandung
- Bekasi
- Cirebon
- Sukabumi

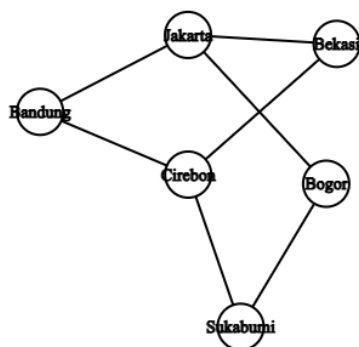
=== HUBUNGAN ANTAR KOTA (EDGE) ===
Bogor <-> Jakarta
Bogor <-> Sukabumi
Jakarta <-> Bandung
Jakarta <-> Bekasi
Bandung <-> Cirebon
Bekasi <-> Cirebon
Cirebon <-> Sukabumi

=== ADJACENCY MATRIX ===
          Bogor   Jakarta   Bandung   Bekasi   Cirebon   Sukabumi
Bogor      0         1         0         0         0         1
Jakarta     1         0         1         1         0         0
Bandung     0         1         0         0         1         0
Bekasi      0         1         0         0         1         0
Cirebon     0         0         1         1         0         1
Sukabumi    1         0         0         0         1         0

=== ANALISIS GRAPH ===
1. Jenis Graph : Undirected Graph
2. Node adalah kota
3. Edge adalah jalan penghubung antar kota
4. Adjacency list lebih cocok digunakan karena lebih sederhana dan hemat memori
5. Graph termasuk sparse graph

```

3. Gambar



4. Analisis adjacency list vs matrix

Pada praktikum ini, graph direpresentasikan menggunakan adjacency list dan adjacency matrix untuk menunjukkan hubungan antar kota pada peta kota. Adjacency list menampilkan daftar kota yang terhubung langsung dengan kota tertentu sehingga lebih mudah dibaca dan lebih hemat memori karena hanya menyimpan hubungan yang ada. Sedangkan adjacency matrix menggunakan tabel berisi angka 0 dan 1 untuk menunjukkan ada atau tidaknya hubungan antar kota, sehingga memudahkan pengecekan koneksi antar node secara cepat. Pada graph yang dibuat, adjacency list lebih cocok digunakan karena jumlah edge tidak terlalu banyak sehingga lebih efisien dan sederhana untuk dipahami, sementara adjacency matrix tetap berguna untuk melihat hubungan antar node dalam bentuk tabel meskipun membutuhkan memori lebih besar jika jumlah node semakin banyak.

5. Kesimpulan singkat

graph dapat digunakan untuk merepresentasikan hubungan antar kota dalam bentuk node dan edge. Node digunakan sebagai simbol kota, sedangkan edge digunakan sebagai jalan penghubung antar kota. Pada praktikum ini digunakan dua representasi graph, yaitu adjacency list dan adjacency matrix. Adjacency list lebih efisien dan mudah dipahami untuk graph dengan jumlah edge sedikit, sedangkan adjacency matrix memudahkan melihat hubungan antar node dalam bentuk tabel. Dengan praktikum ini, pemahaman mengenai konsep graph, node, edge, adjacency list, dan adjacency matrix menjadi lebih baik.